

Festival

Nayra đang ở một lễ hội và tham gia một trò chơi có một giải thưởng lớn là chuyển đi đến Laguna Colorada. Trò chơi bao gồm việc sử dụng token để mua phiếu khuyến mại. Mua một phiếu khuyến mại có thể được nhận thêm token. Mục tiêu là lấy được càng nhiều phiếu khuyến mại càng tốt.

Cô ấy bắt đầu trò chơi này với A token. Có N phiếu khuyến mại, được đánh số từ 0 đến $N - 1$. Nayra phải trả $P[i]$ token ($0 \leq i < N$) để mua phiếu khuyến mại i (và cô ấy phải có ít nhất $P[i]$ token trước khi mua). Cô ấy chỉ có thể mua mỗi phiếu khuyến mại nhiều nhất một lần.

Hơn nữa, mỗi phiếu khuyến mại i ($0 \leq i < N$) được gán một **loại**, được biểu thị bằng $T[i]$, là số nguyên **giữa 1 và 4, bao gồm cả hai đầu mút**. Sau khi Nayra mua phiếu khuyến mại i , số lượng token còn lại của cô ấy sẽ được nhân với $T[i]$. Về mặt hình thức, nếu cô ấy có X token tại một thời điểm nào đó của trò chơi và mua phiếu khuyến mại i (yêu cầu $X \geq P[i]$) thì cô ấy sẽ có $(X - P[i]) \cdot T[i]$ token sau khi mua.

Nhiệm vụ của bạn là xác định Nayra nên mua phiếu khuyến mại nào và theo thứ tự nào, để tối đa hóa tổng số **phiếu khuyến mại** mà cô ấy có vào lúc kết thúc trò chơi. Nếu có nhiều hơn một phương án mua hàng đạt được điều này, bạn có thể đưa ra bất kì phương án nào trong số đó.

Chi tiết cài đặt

Bạn cần cài đặt hàm sau.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : số lượng token ban đầu mà Nayra có.
- P : một mảng có kích thước N cho biết giá của các phiếu khuyến mại.
- T : một mảng có kích thước N cho biết loại của các phiếu khuyến mại.
- Hàm này được gọi đúng một lần cho mỗi trường hợp thử nghiệm.

Hàm này sẽ trả về một mảng R , cho biết các giao dịch mua của Nayra như sau:

- Kích thước của R phải là số lượng phiếu khuyến mại nhiều nhất mà cô ấy có thể mua.
- Các phần tử của mảng là chỉ số của các phiếu khuyến mại mà cô ấy cần mua, theo thứ tự thời gian. Tức là cô ấy mua phiếu khuyến mại $R[0]$ đầu tiên, phiếu khuyến mại $R[1]$ thứ hai, v. v.

- Tất cả các phần tử của R phải là duy nhất.

Nếu không thể mua được phiếu khuyến mại, R phải là một mảng rỗng.

Các ràng buộc

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ với mỗi i sao cho $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ với mỗi i sao cho $0 \leq i < N$.

Các subtask

Subtask	Điểm	Các ràng buộc bổ sung
1	5	$T[i] = 1$ với mỗi i sao cho $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ với mỗi i sao cho $0 \leq i < N$.
3	12	$T[i] \leq 2$ với mỗi i sao cho $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra có thể mua tất cả N phiếu khuyến mại (theo một thứ tự nào đó).
6	16	$(A - P[i]) \cdot T[i] < A$ với mỗi i sao cho $0 \leq i < N$.
7	18	Không có ràng buộc nào thêm.

Các ví dụ

Ví dụ 1

Xét lời gọi hàm sau.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Ban đầu Nayra có $A = 13$ token. Cô ấy có thể mua 3 phiếu khuyến mại theo thứ tự hiển thị bên dưới:

Phiếu khuyến mại đã mua	Giá phiếu khuyến mại	Loại phiếu	Số lượng token sau khi mua
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

Trong ví dụ này, Nayra không thể mua nhiều hơn 3 phiếu khuyến mại, và phương án mua hàng được mô tả ở trên là cách duy nhất để cô ấy có thể mua được 3 phiếu. Do đó, hàm này sẽ trả về $[2, 3, 0]$.

Ví dụ 2

Xét lời gọi hàm sau.

```
max_coupons(9, [6, 5], [2, 3])
```

Trong ví dụ này, Nayra có thể mua cả hai phiếu khuyến mại theo bất kì thứ tự nào. Do đó, hàm này sẽ trả về $[0, 1]$ hoặc $[1, 0]$.

Ví dụ 3

Xét lời gọi hàm sau.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

Trong ví dụ này, Nayra có một token, không đủ để mua bất kì phiếu khuyến mại nào. Do đó, hàm này sẽ trả về $[]$ (một mảng rỗng).

Trình chấm mẫu

Định dạng dữ liệu vào:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Định dạng kết quả ra:

```
S
R[0] R[1] ... R[S-1]
```

Trong đó, S là kích thước của mảng R được trả về bởi hàm `max_coupons`.